Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 04 Notes
Control Flow + Arrays + Strings

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Program control statements (selection + loops) . . . 3

7.1.1 `if/else` . . . 3

7.1.2 `switch` . . . 4

7.1.3 Loops . . . 4

7.2 Arrays (1D/2D), types of arrays, common patterns . . . 4

7.2.1 1D arrays . . . 4

7.2.2 2D and jagged arrays . . . 4

7.3 Strings and `StringBuilder` . . . 4

7.3.1 String immutability . . . 4

7.3.2 `==` vs `equals` . . . 5

7.3.3 When to use `StringBuilder` . . . 5

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: Grades (control flow + arrays + strings) **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **7**

13 Self-Study Practice (no solutions) **7**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals
- **Main new ideas:** Program control statements (selection + loops), Arrays (1D/2D), types of arrays, common patterns, Strings and `StringBuilder`
- **Course thread:** Use Control Flow + Arrays + Strings to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

2 Lecture Overview

This lecture teaches you how to **control program flow** (decisions + loops) and work with two core data structures: **arrays** (for many values of the same type) and **strings** (text).

These three topics appear in almost every lab/program you will write.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Types and operators**: Conditions, loop bounds, indices, and string logic depend on the previous lecture. Main reference: `Unit 01 / Lecture 03 slides/notes`.
- **Program flow**: Control statements only make sense once you already know the default top-to-bottom execution order. Main reference: `Unit 01 / Lecture 02 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Use `if/else`, `switch`, and loops to control program flow.
- Create/traverse 1D and 2D arrays and solve simple array problems.
- Explain `String` immutability and use `StringBuilder` when needed.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Program control statements (selection + loops) → Arrays (1D/2D), types of arrays, common patterns → Strings and `StringBuilder`.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Selection**: choosing one path among many (`if`, `switch`).
- **Iteration**: repeating work until a condition changes (`for`, `while`).
- **Array**: fixed-size contiguous collection of same-type elements.
- **Index**: position in array (0-based in Java).
- **Immutable**: cannot be changed after creation (e.g., `String`).
- **StringBuilder**: mutable sequence of characters (efficient for repeated concatenation).

7 Detailed Notes

7.1 Program control statements (selection + loops)

7.1.1 `if/else`

Use `if` when you have conditions based on boolean expressions.

Good practice:

- keep conditions readable (break complex conditions into variables),
- avoid deeply nested `if` blocks (use early returns in methods later).

7.1.2 `switch`

Use `switch` when one variable is compared against many constant values. It often makes menu-driven programs readable.

Pitfall: forgetting `break` can cause fall-through (sometimes intended, often a bug).

7.1.3 Loops

- `for`: when you know the number of iterations (or array traversal).
- `while`: when you loop until a condition changes.
- `do-while`: when the body must run at least once.

Pitfall: off-by-one errors. Remember arrays are 0-based and last index is `length - 1`.

7.2 Arrays (1D/2D), types of arrays, common patterns

7.2.1 1D arrays

Create:

- `int[] a = new int[5];` (all zeros initially)
- `int[] b = {1,2,3};` (initialized)

Access:

- `a[i]` for element, `a.length` for size (not a method).

Common patterns:

- sum/average, max/min, linear search, counting, reversing.

7.2.2 2D and jagged arrays

2D array:

- `int[][] m = new int[rows][cols];`

Jagged array (rows can have different lengths):

- `int[][] j = new int[3][];` then allocate each row separately.

Pitfall: always check `m[i].length` for column length in jagged arrays.

7.3 Strings and `StringBuilder`

7.3.1 String immutability

In Java, `String` is immutable: operations like `toLowerCase()` or `trim()` return a new string.

Why it matters:

- Safer sharing: if two variables reference the same string, it can't be modified unexpectedly.
- Performance implication: repeated concatenation in loops creates many temporary objects.

7.3.2 `==` vs `equals`

Use:

- `==` to compare references (same object or not),
- `equals()` to compare string content.

7.3.3 When to use `StringBuilder`

If you build a string repeatedly (especially in a loop), use `StringBuilder` and `append()`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

8 Running Project Connection

Use Control Flow + Arrays + Strings to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Writing off-by-one loop boundaries.
- Comparing String values with == instead of an equality method.
- Treating String as mutable and expecting in-place updates.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

10 Worked Example: Grades (control flow + arrays + strings)

```java
import java.util.Scanner;

public class GradeStats {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
        int[] marks = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Marks for student " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
        }

        int max = marks[0];
        int sum = 0;
        for (int x : marks) {
            sum += x;
            if (x > max) max = x;
        }

        double avg = sum / (double) n;
        System.out.printf("Max=%d, Avg=%.2f%n", max, avg);

        String result = (avg >= 40) ? "PASS" : "NEEDS IMPROVEMENT";
        System.out.println("Class status: " + result);

        sc.close();
    }
}
```

What to notice:

- Off-by-one handling in prompts: `i+1` for human-friendly numbering.
- Enhanced for-loop (`for (int x : marks)`) to traverse arrays.
- Casting for average: `sum / (double)n` to avoid integer division.

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Program control statements (selection + loops)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. What is the last valid index of an array of length 10? **Answer:** 9.
2. Should we use `==` or `equals` to compare strings by content? **Answer:** `equals`.
3. Which loop runs at least once: `while` or `do-while`? **Answer:** `do-while`.

13 Self-Study Practice (no solutions)

1. Write a program to find the second largest element in an array.
2. Write a program to check if a string is a palindrome (ignore spaces and case).
3. Create a 2D matrix and compute row-wise sums.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why is `String` immutable and why does it matter?

Sample answer: Immutability makes strings safe to share and helps with security and caching. It also means operations create new objects, so repeated concatenation in loops can be inefficient.

Q2. How do you avoid off-by-one errors in array loops?

Sample answer: Remember Java arrays are 0-based and the last index is `length - 1`. Use loop conditions like `i < a.length` rather than `i <= a.length`.

Q3. Why is String immutability useful?

Sample answer: It makes String values safer to share and easier to reason about across APIs and larger programs.